Georgia Tech

INTERRUPTS AND CONCURRENCY (PART 1) (+ SWAPPING WRAP-UP)

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

ADMINISTRATION

- Lab2: Due 1 week from Friday (Feb 27)
  - I recommend to be `code complete` and submitting your full lab code by end of week 2/3
  - It **will** take week 3/3 to debug!
    - Please leave a week to do it
  - This is 10 days from today
    - 50 possible submission opportunities (5*10)

Lab 2	Expand the assignment to display section dates.	10	50%

CODE SHARING WITH PARTNER

- Initialize **private** repo on github.gatech.edu
- git clone xv6-public #clone xv6 to local drive
- git checkout lab2 #check out the lab branch
- git remote add lab2private private_github_repo_url
- git remote –v # verify lab2private remote was added
- git push lab2private lab2

RECAP QUESTION

- How much did you cover disk swapping in 2200?
  - Did you cover the “clock” page eviction algorithm?
    - How do we choose which pages to evict?

VIRTUAL MEMORY TIPS AND TRICKS

- Memory Backing
  - Is our physical memory limited by our RAM? NO
  - Do our virtual mappings have to be backed by RAM? NO
    - *Swapping* – saving memory to disk when we run out of RAM – "swap" some mem. to disk
    - *Memory Mapping* – mapping file information into virtual addresses, and writing it back to the file

VIRTUAL MEMORY TIPS AND TRICKS

- Swapping
  - How do we choose which pages to evict to disk?
    - What is the ideal page to evict to disk?
    - Ultimately, we want the page that will be accessed latest
    - We cannot know this, so we prefer Least Recently used (LRU) approximation
    - But, least recently used is expensive to track…

Random?

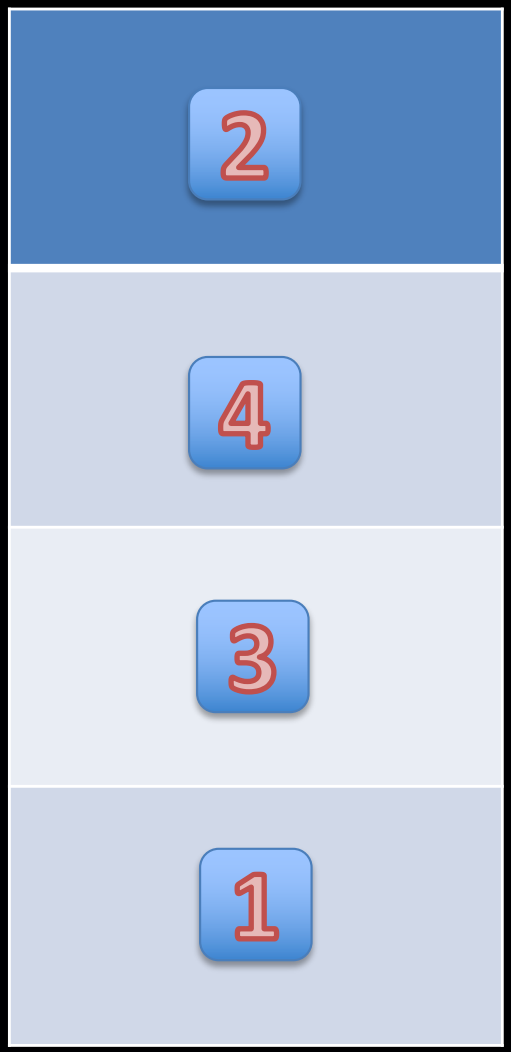
Belady?

How do we track LRU for pages?

TRACKING LRU

- What would it take to track information needed by LRU
  - When the page was accessed (timestamp information)
- How expensive is it?
  - Needed for each access, including reads and writes
  - Unnecessary overhead, even with infinite memory!
- Complexity of tracking LRU timestamp info
  - Worst case linear?
  - O(n) → O(logn) → O(1) , update d.s. on each memory access
- Worst overhead : trap on every access
  - page fault for every load and store

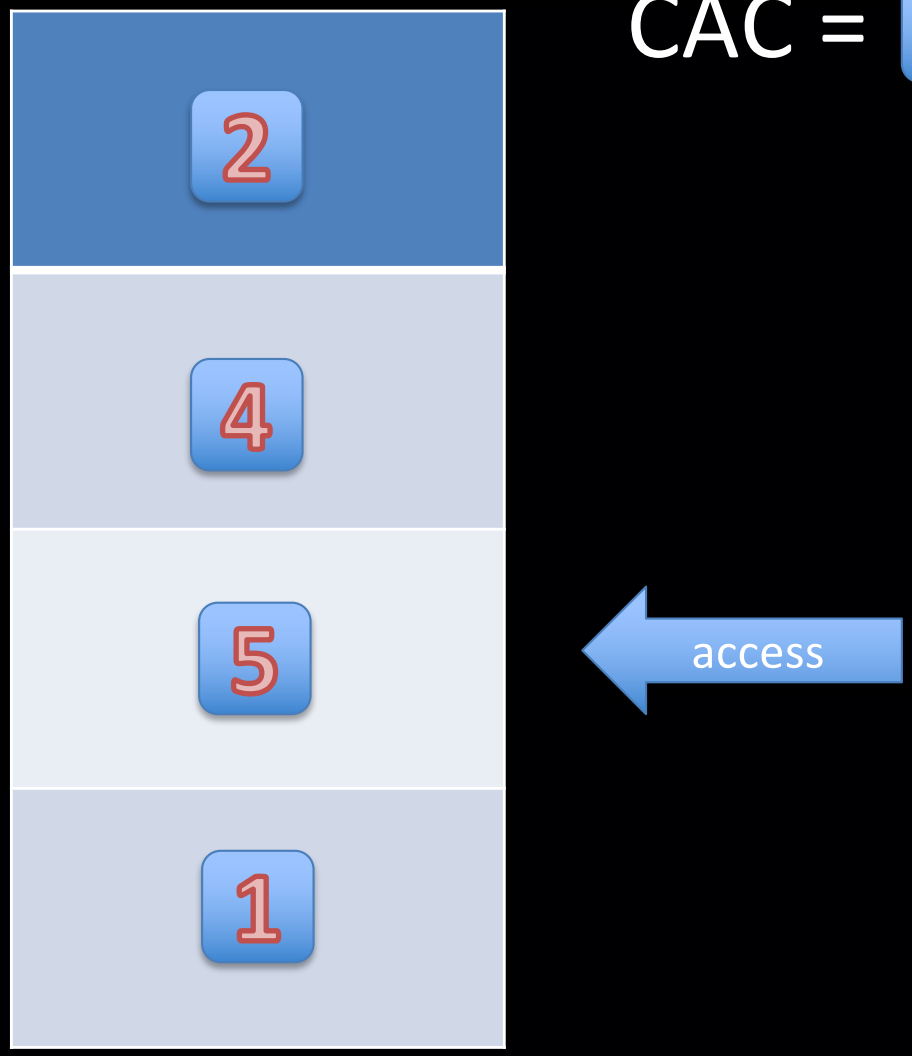
CLOCK ALGORITHM:



CAC = 5

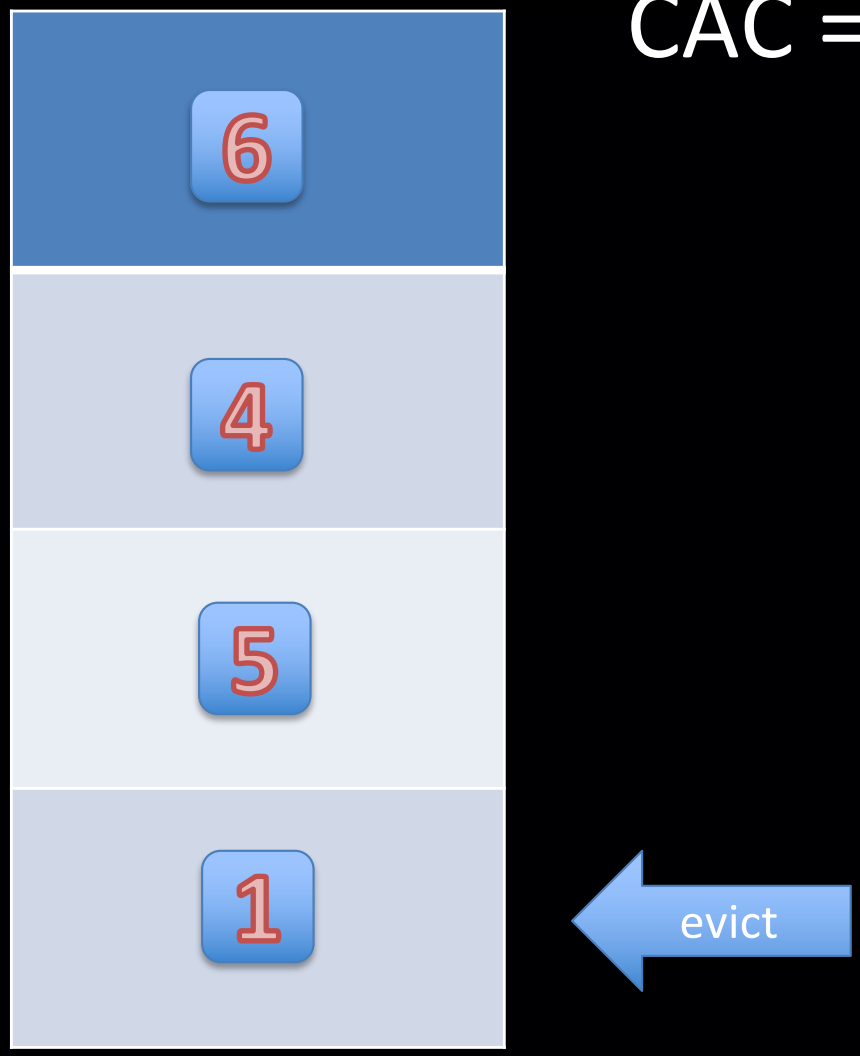
CLOCK ALGORITHM:

CAC = 6

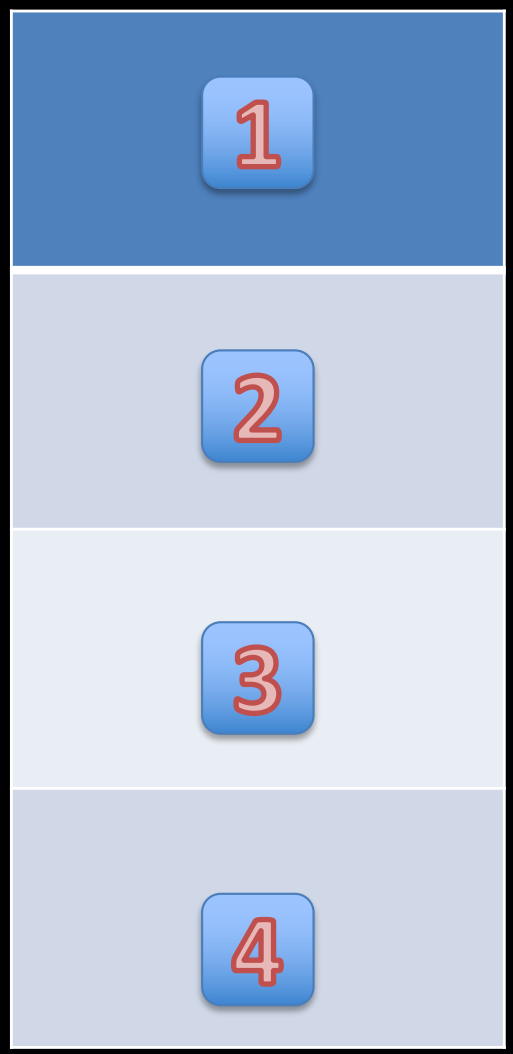


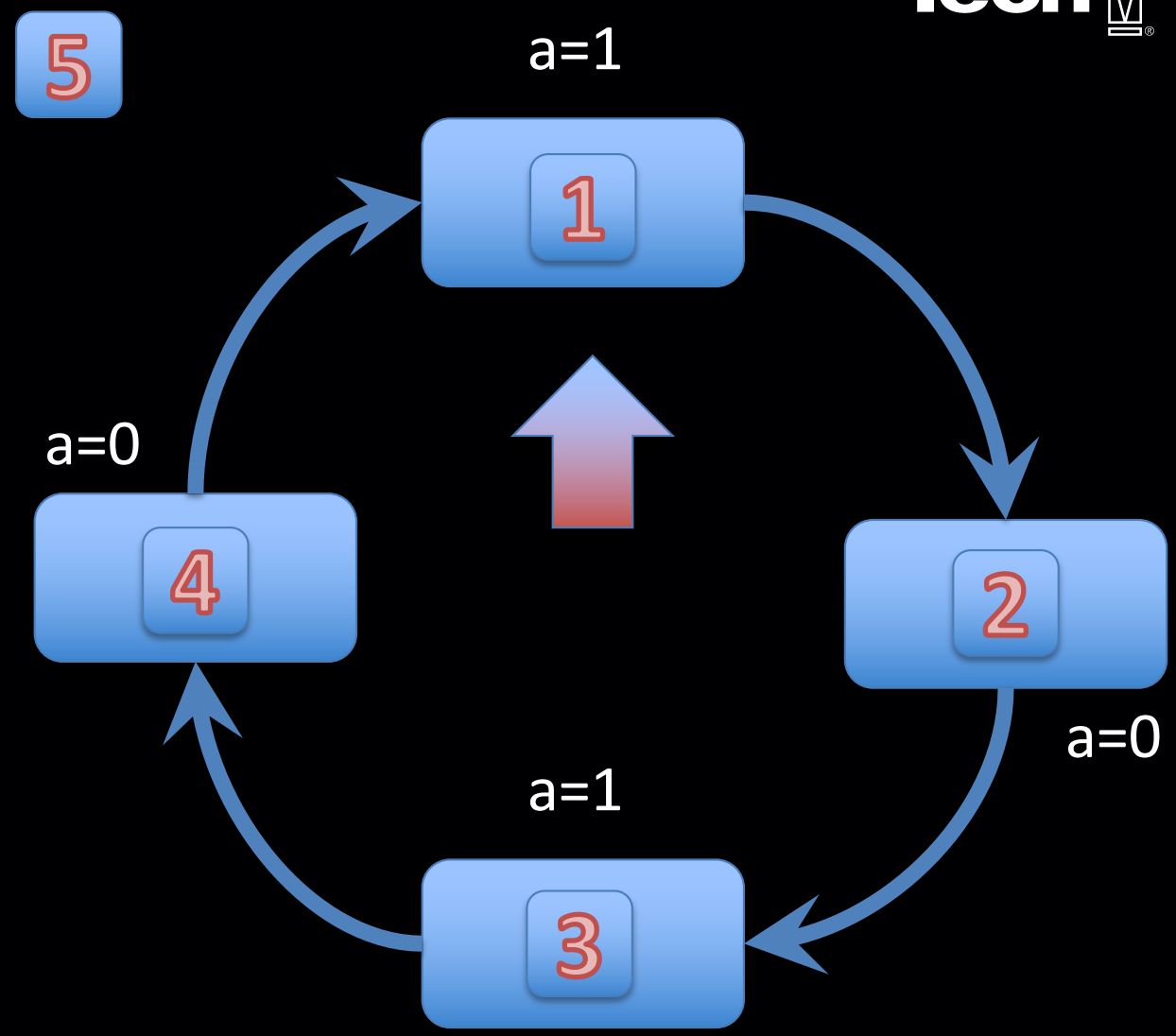
CLOCK ALGORITHM:

CAC = 7

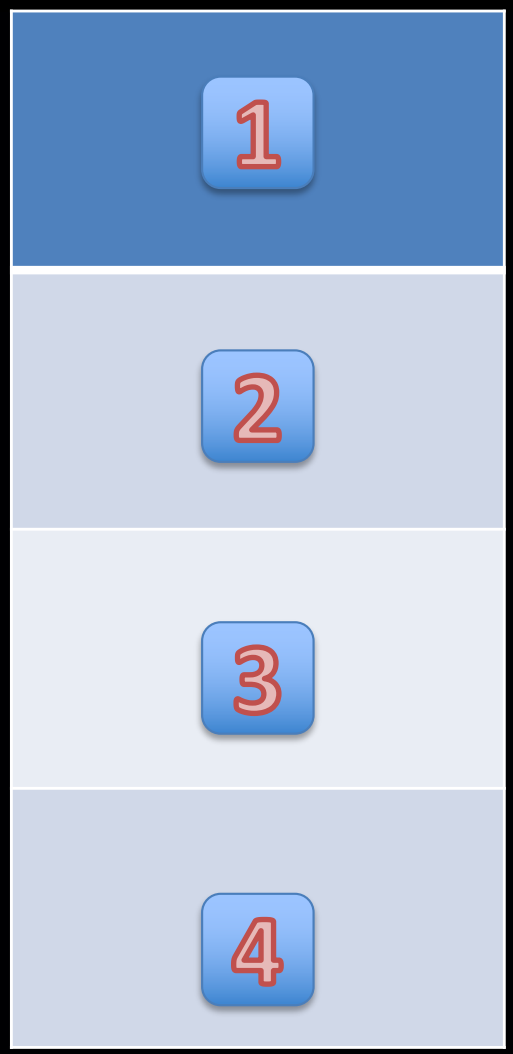


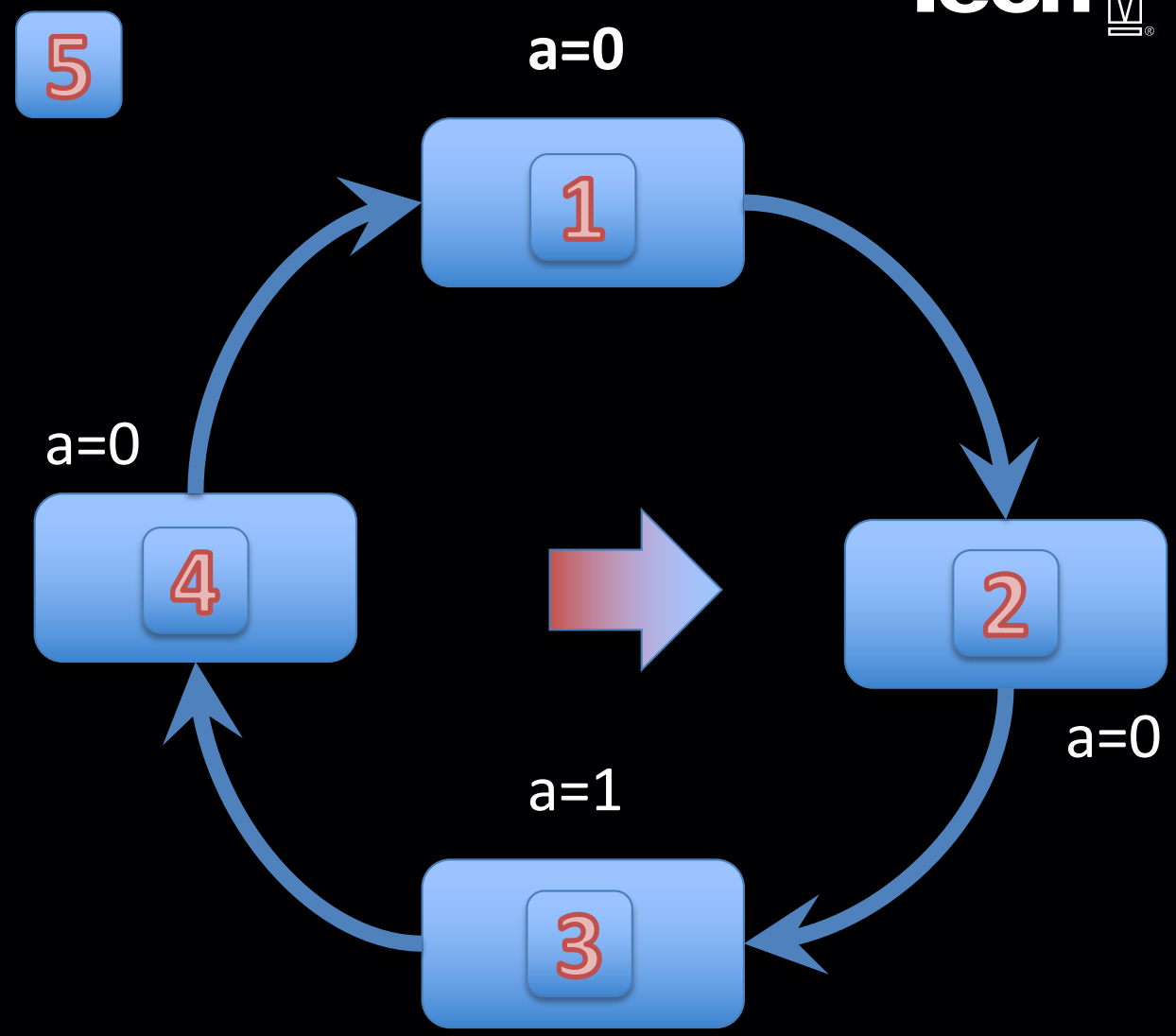
CLOCK ALGORITHM:



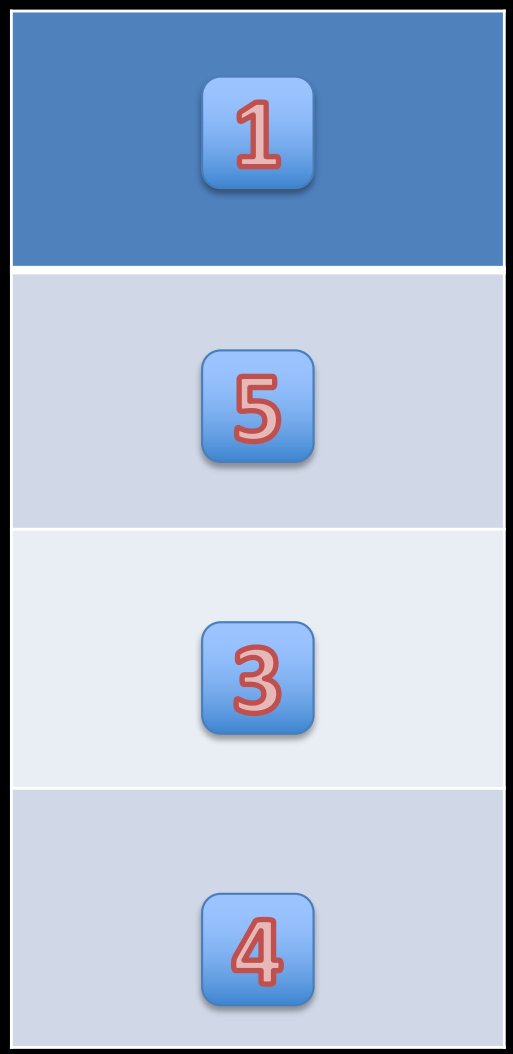


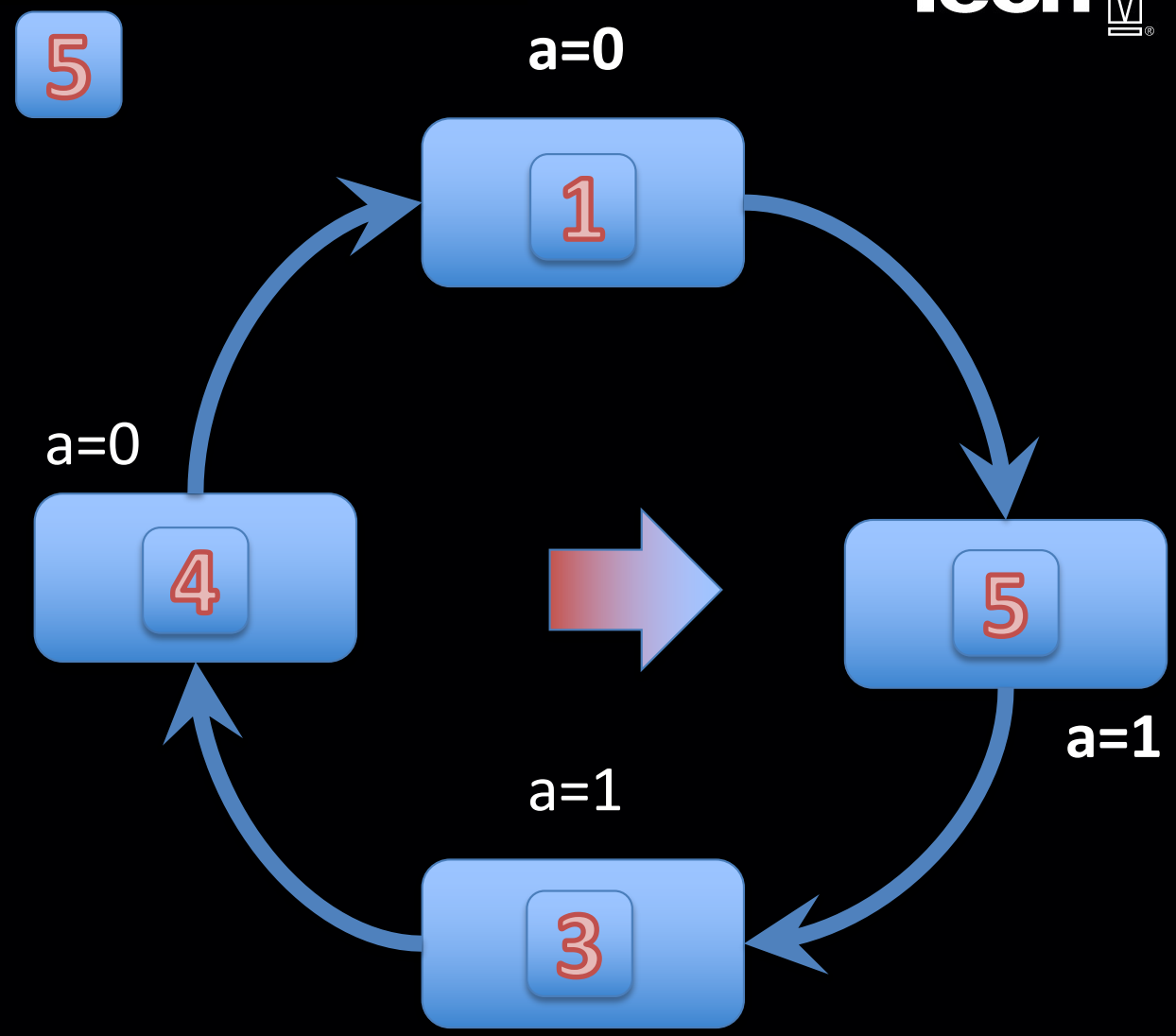
CLOCK ALGORITHM:





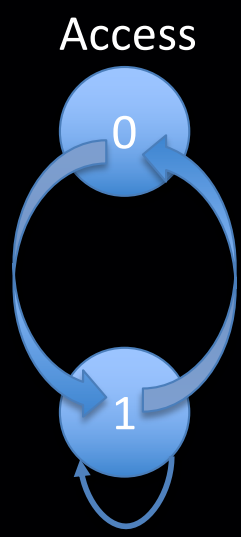
CLOCK ALGORITHM:



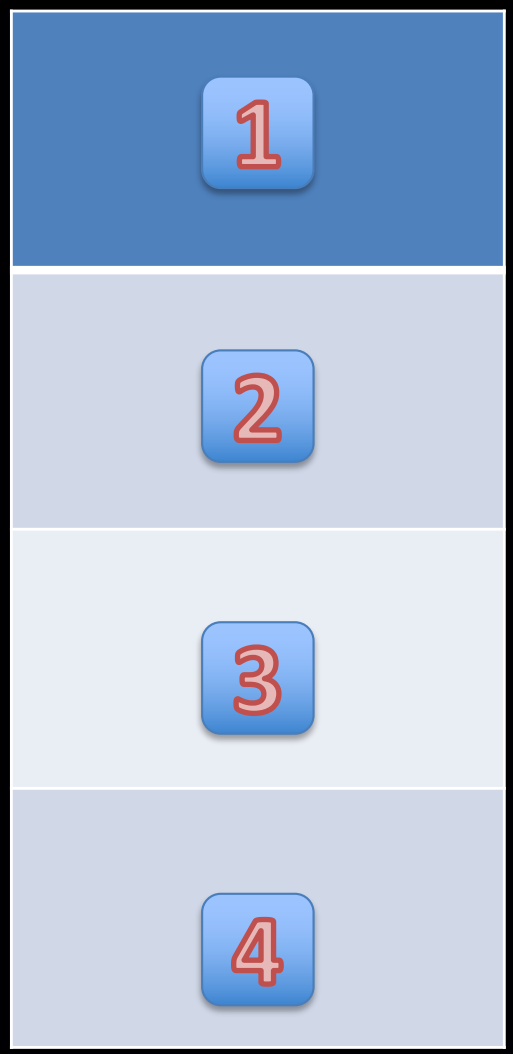


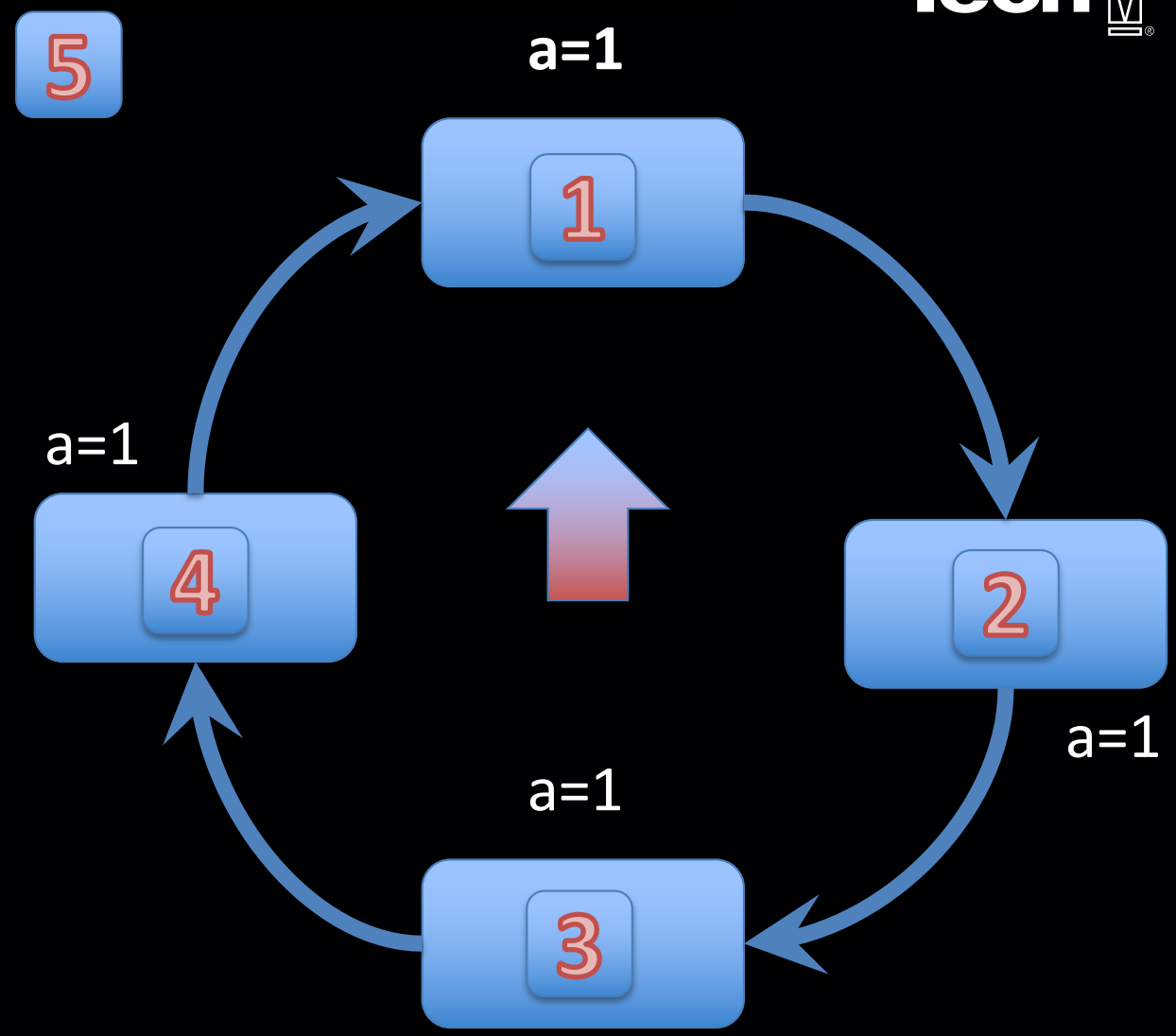
CLOCK ALGORITHM

- Given this clock algo, when do I need to trap on the page read?
  - When do I need notification that a process has accessed a page?
  - Is it every single read and write?
- We need to know when to switch that access bit from 0 to 1
- When a==0, we want PTE_P to be equal to 0 as well
- What's the worst case performance of this clock algo?

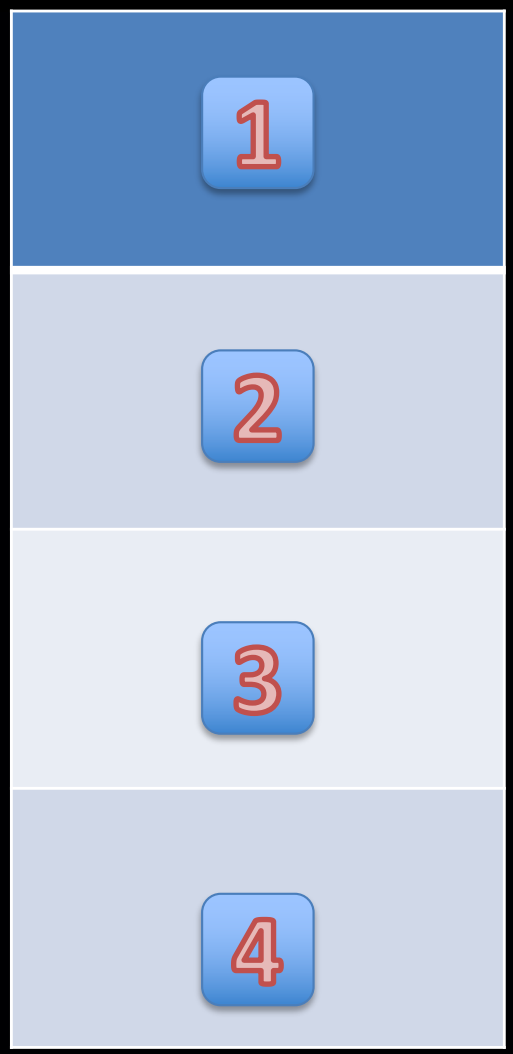


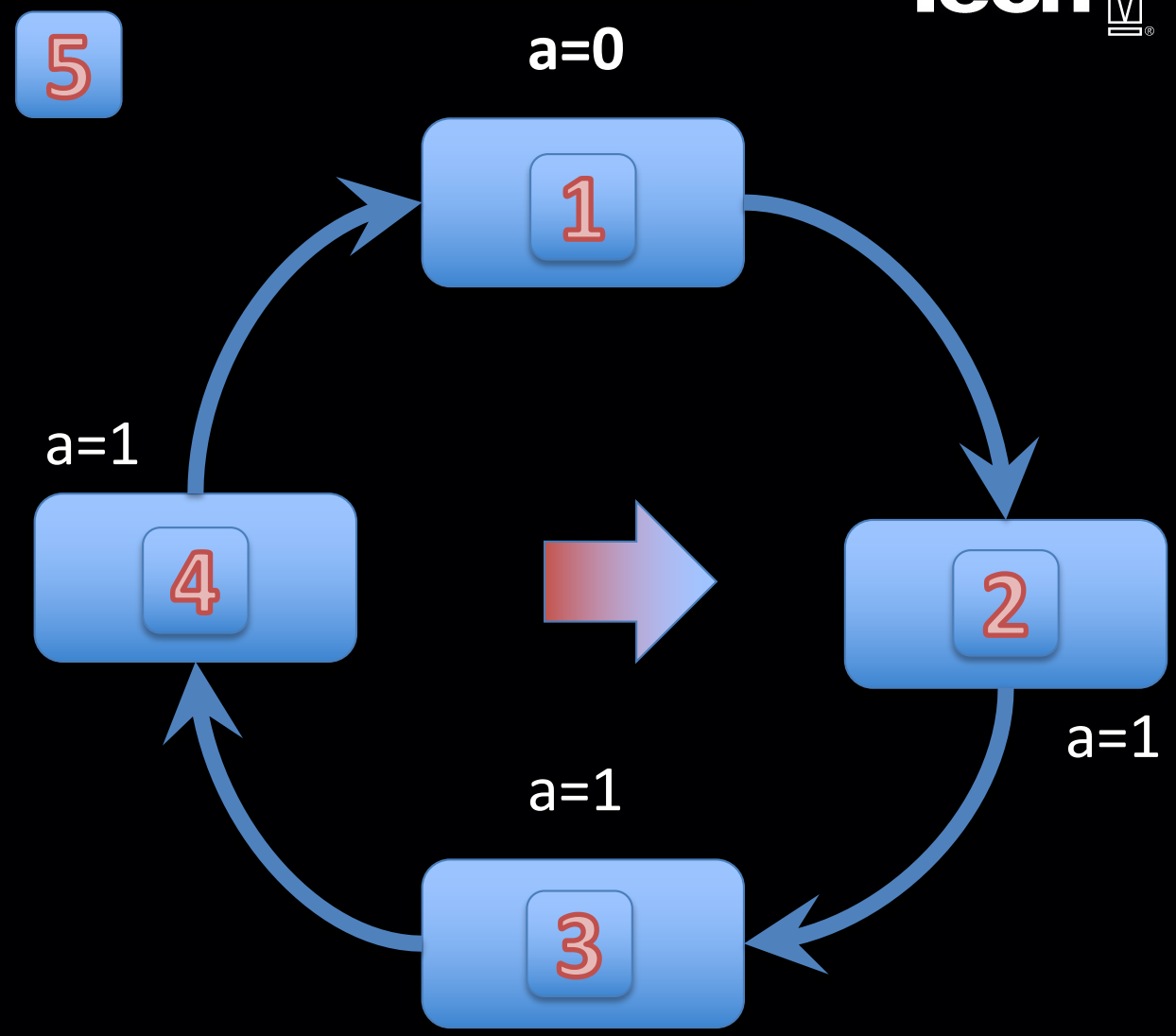
CLOCK ALGORITHM: WORST CASE ANALYSIS



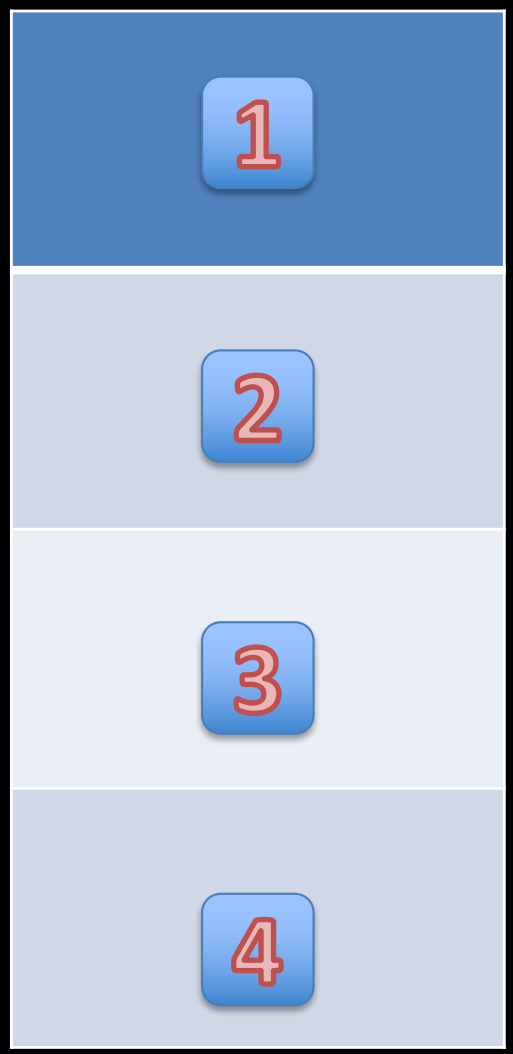


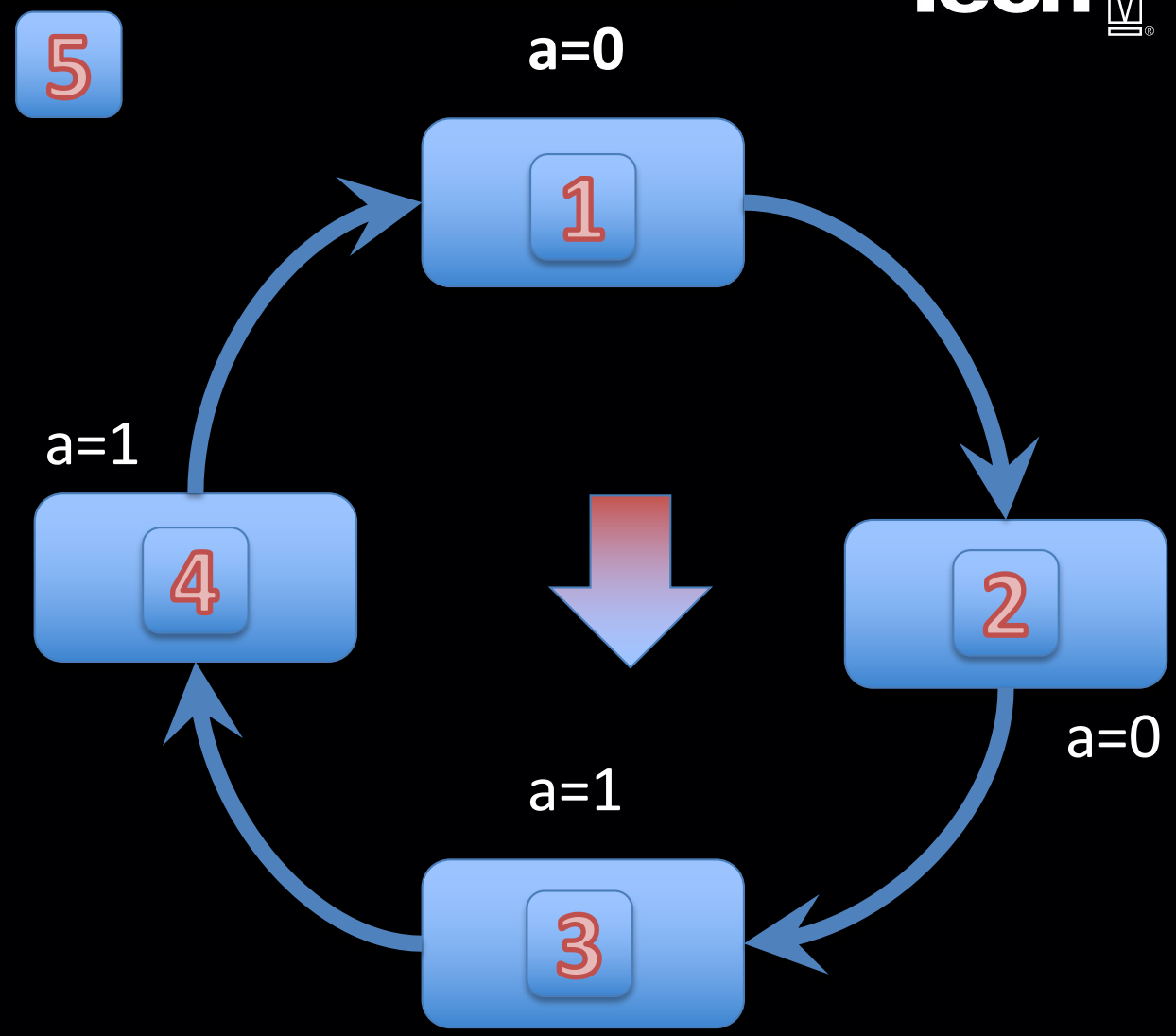
CLOCK ALGORITHM: WORST CASE ANALYSIS



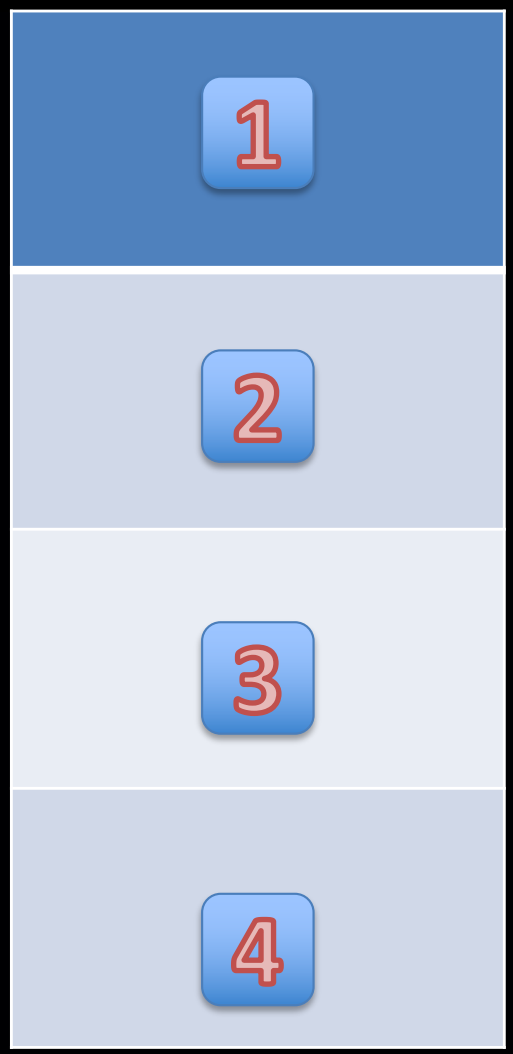


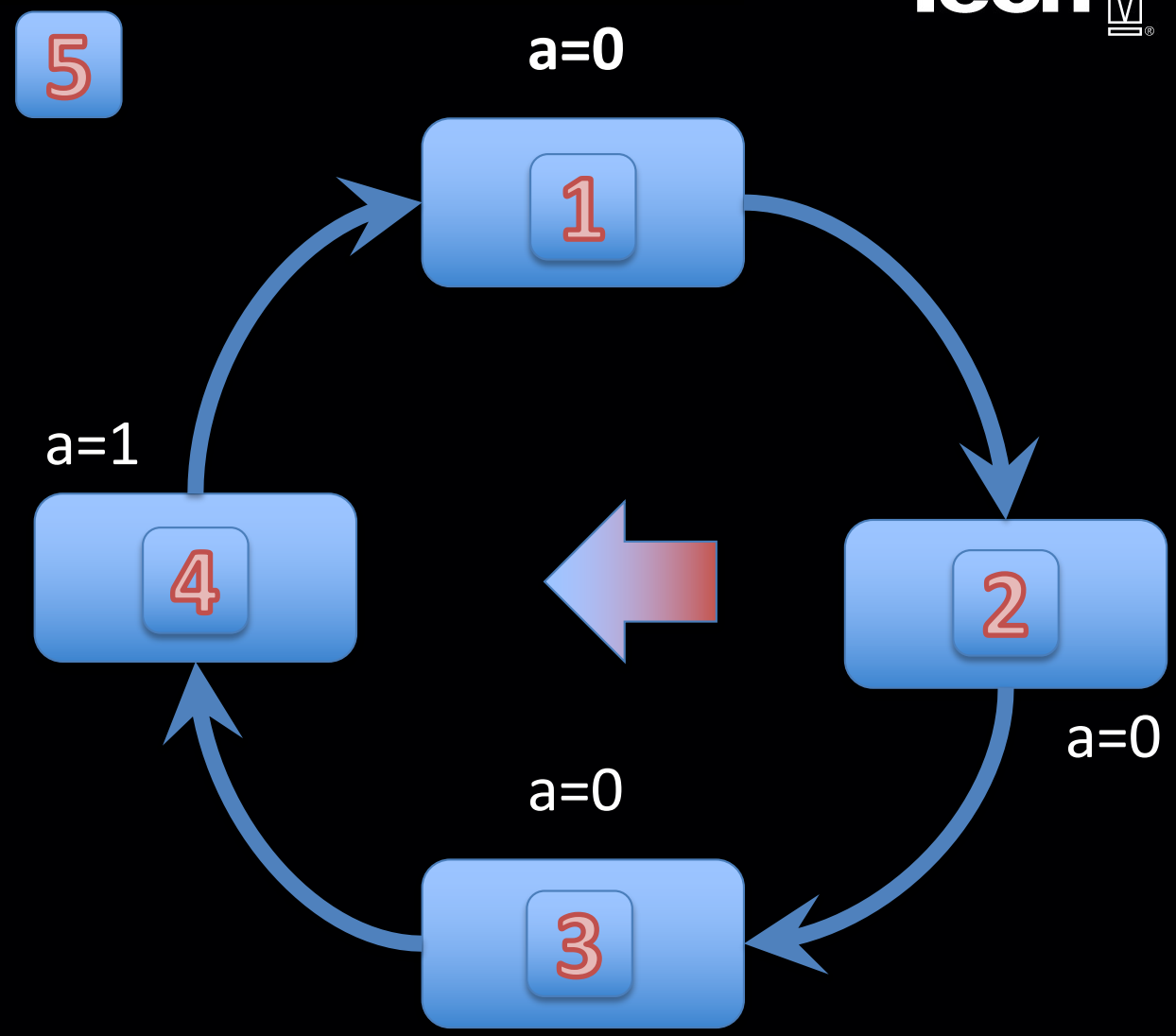
CLOCK ALGORITHM: WORST CASE ANALYSIS



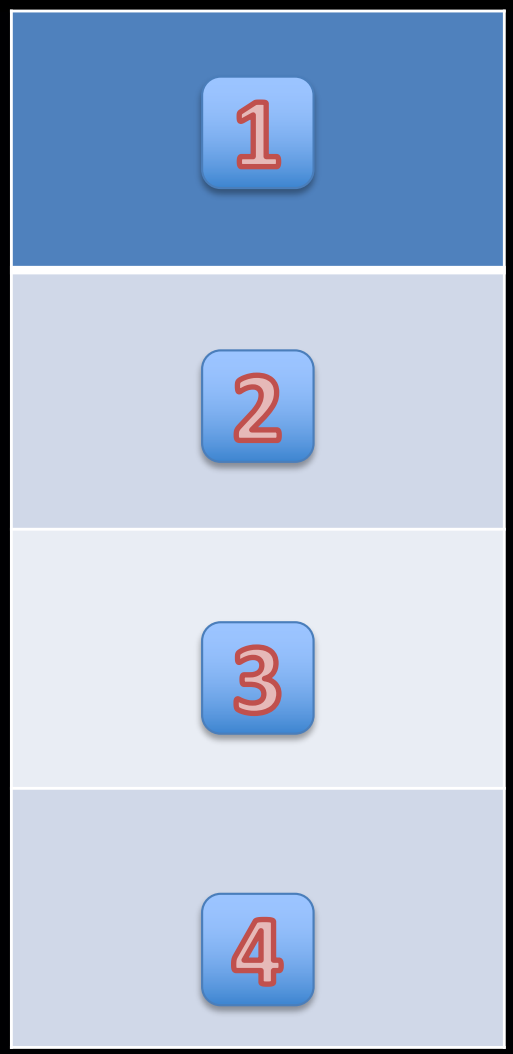


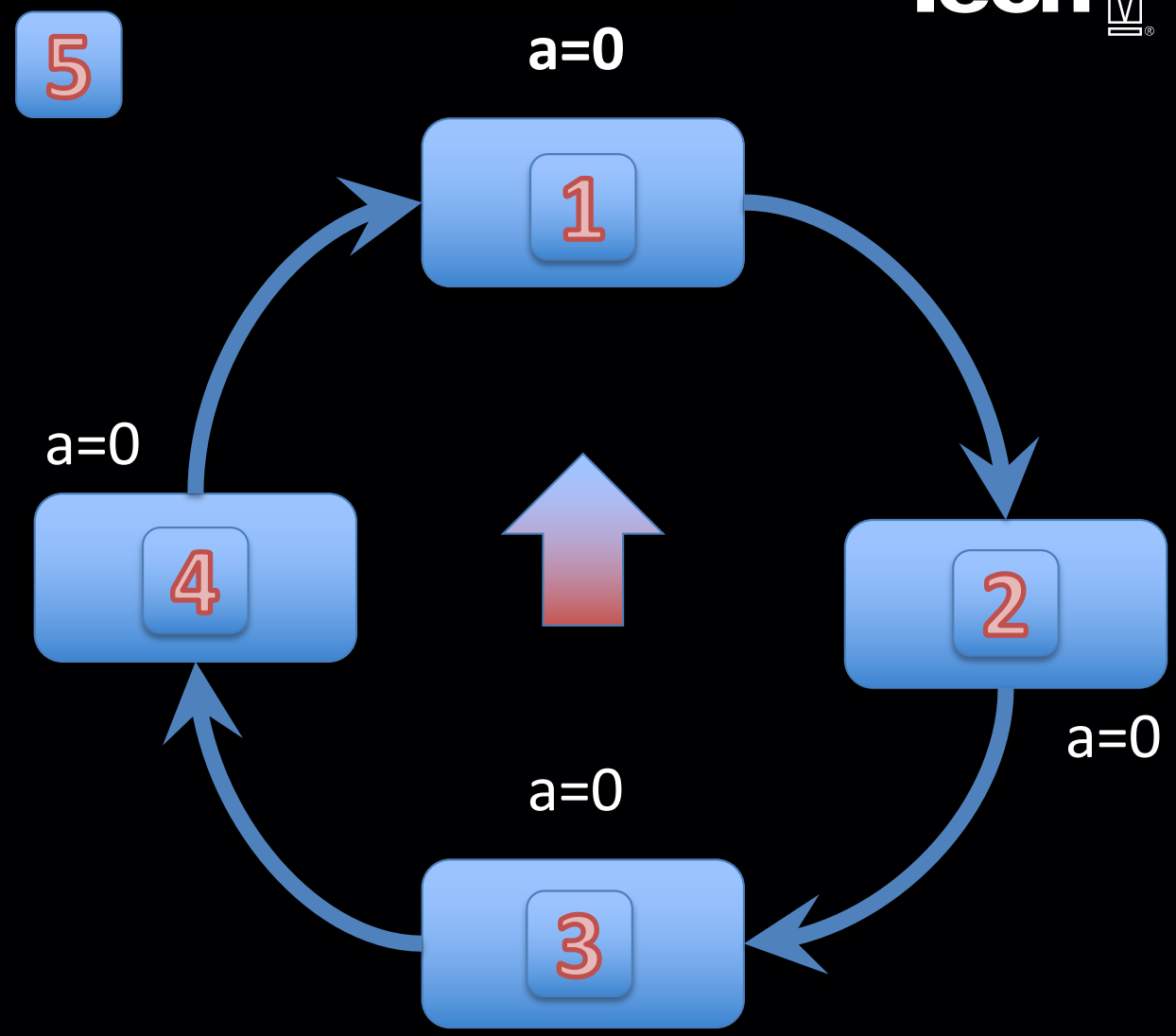
CLOCK ALGORITHM: WORST CASE ANALYSIS



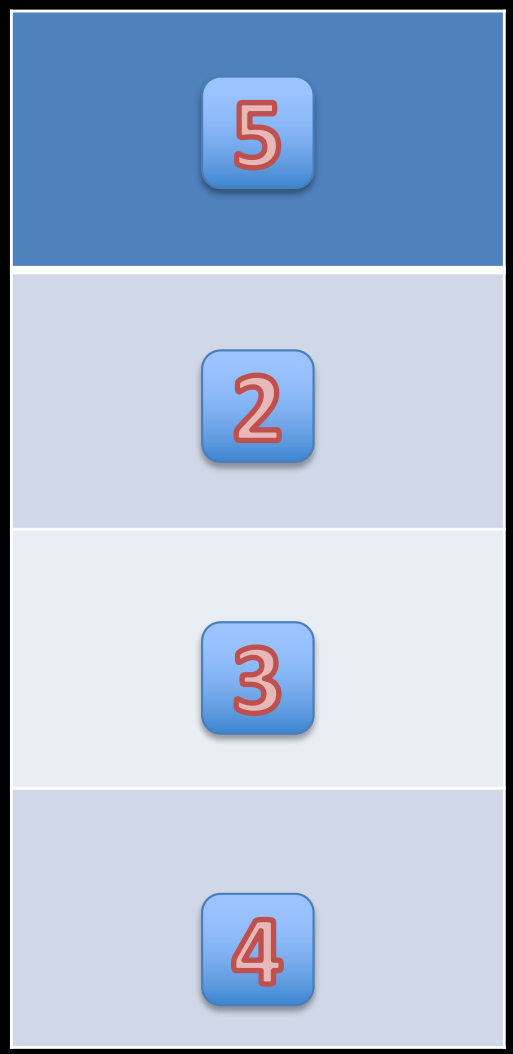


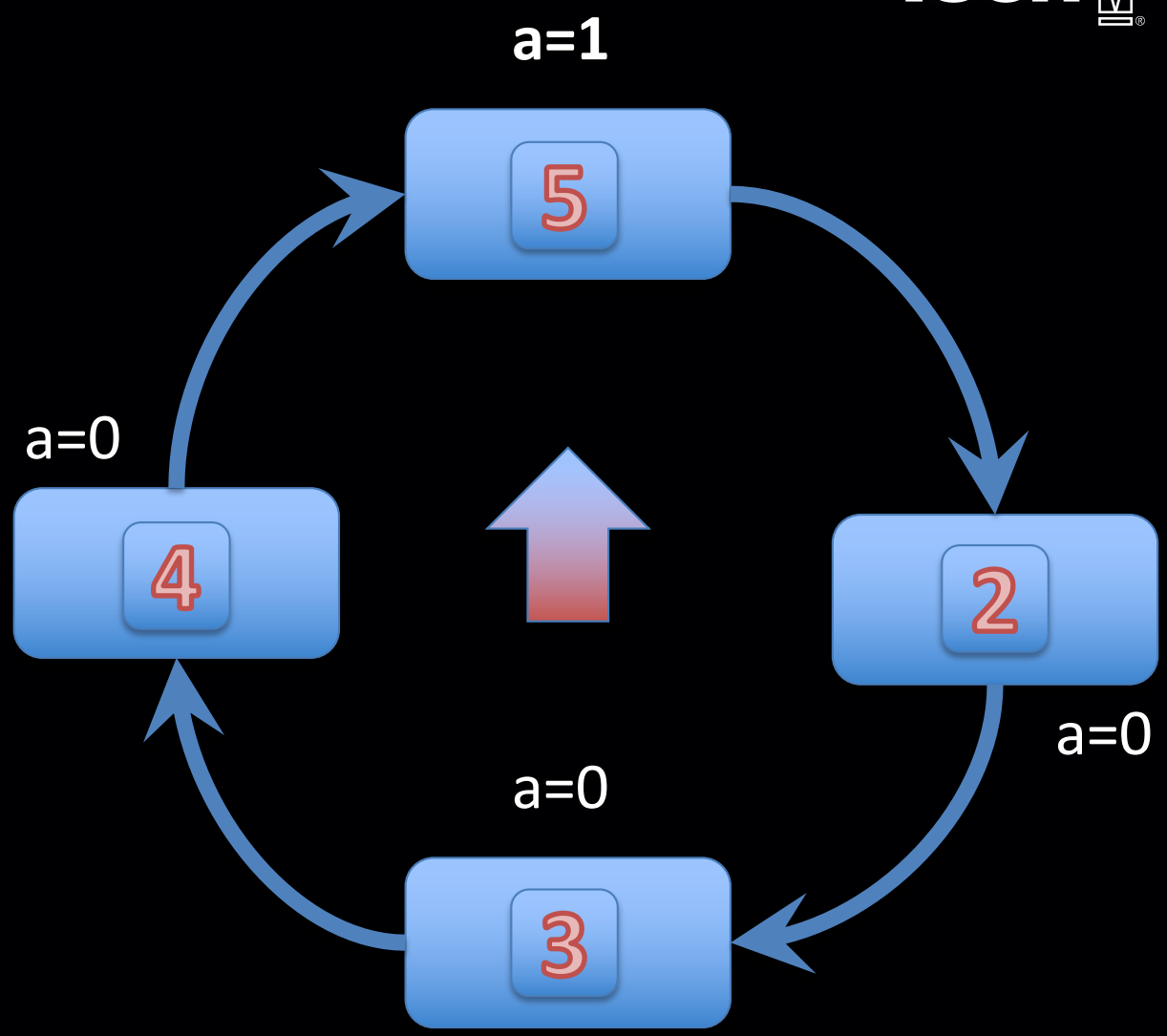
CLOCK ALGORITHM: WORST CASE ANALYSIS





CLOCK ALGORITHM: WORST CASE ANALYSIS





VIRTUAL MEMORY TIPS AND TRICKS

- Swapping
  - How do we choose which pages to evict to disk?
    - Clock Algorithm:
      - Put pages in circular list
      - Mark pages as “accessed” when read/written

      - On evict, clock iterates pages in the queue.
      - If the page is accessed, mark “clean” (unaccessed)
        - Clear access bit (a=0)
      - If the page is unaccessed, evict

Georgia Tech

INTERRUPTS AND CONCURRENCY

CREATING THE NEXT®

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- How do I handle unpredictable events? (e.g. keyboard inputs, network data)

- What things do we need interrupts for?
  - *Exceptions*: ***Paging***, math & hard errors – Power, Memory
  - *Hardware*: Timers, buffers, io-devices
  - *Debugging*: software-breakpoints
  - *Communication*: Inter-processor-interrupts
  - *Control* Features: Watchdog, Non-Maskable

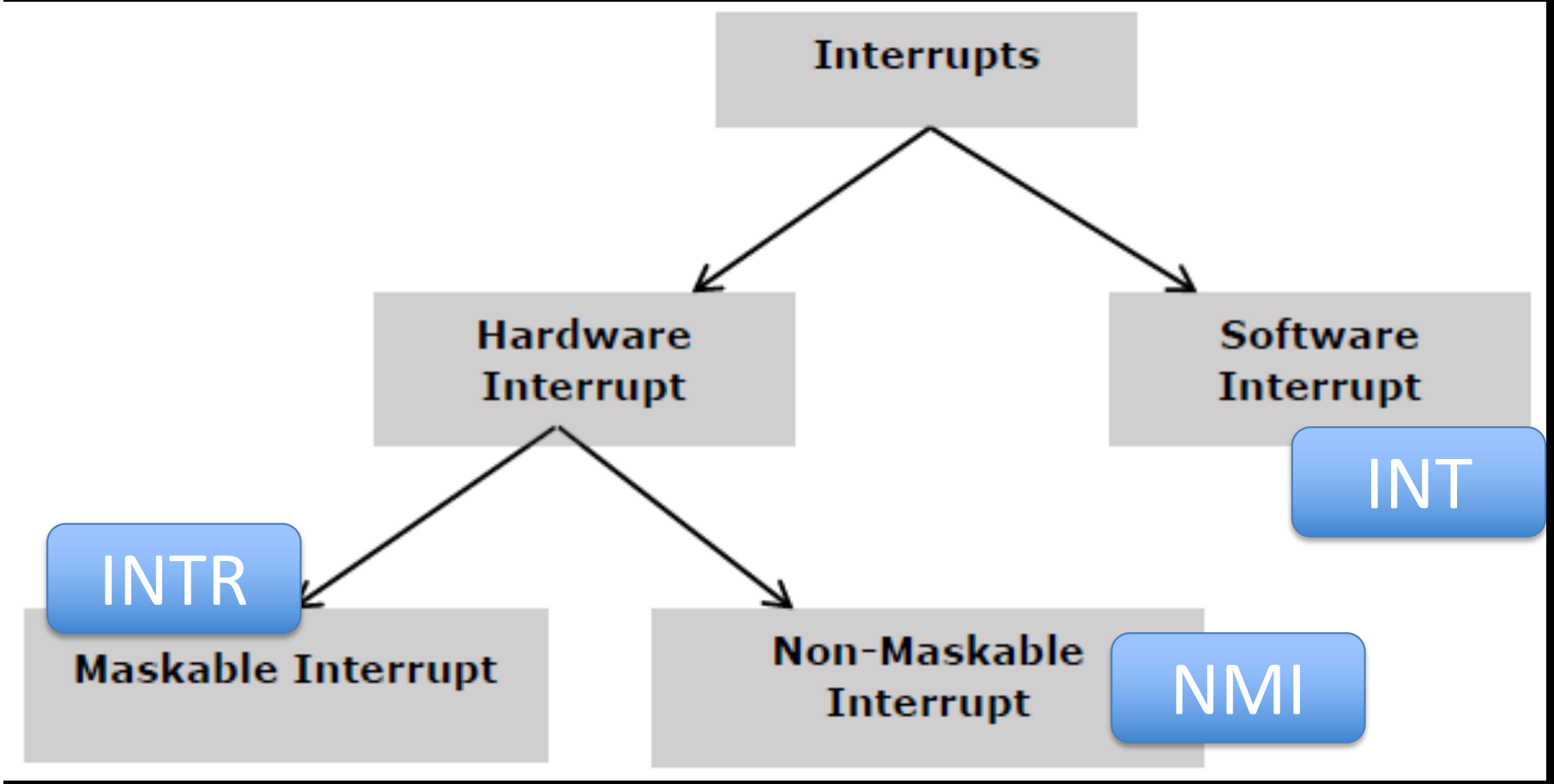
SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- Thought experiment:
- Imagine building a keyboard driver without interrupts…
  - Would have to keep checking for events (called *polling*)
    - Consider: frequency of events
      - frequent
      - infrequent
    - Disadvantages
      - Performance
      - Correctness

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- Every Exception/Interrupt type is assigned a number its vector
- When an interrupt occurs, the vector determines what code is invoked to handle the interrupt
- Example:
  - vector 14 -> page fault handler (lab2)
  - vector 32 -> clock handler -> scheduler (lab3)

INTERRUPT OVERVIEW



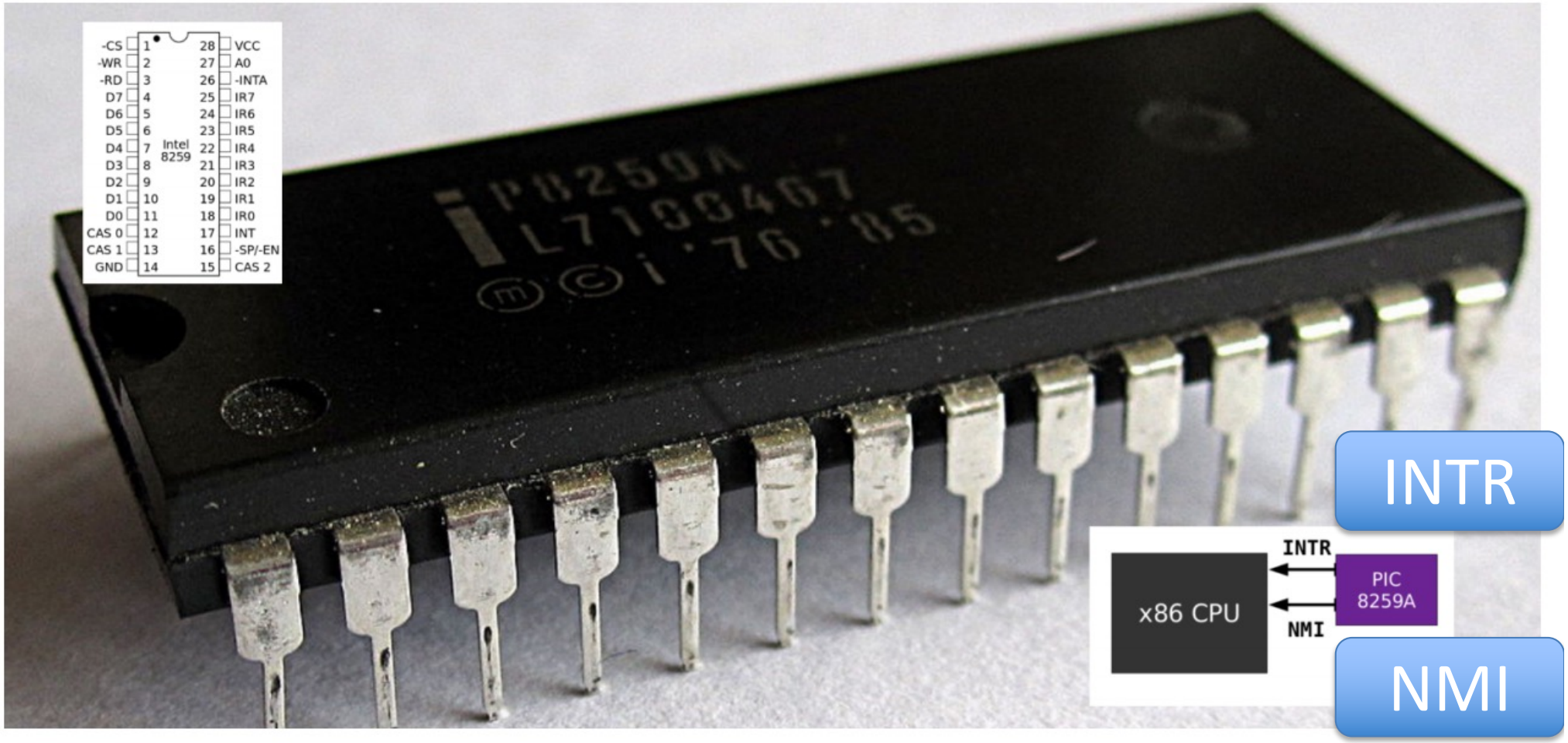
Source: www.tutorialspoint.com/microprocessor/microprocessor_8086_interrupts.htm

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- NMI: Non-Maskable Interrupts NMI
  - Never ignored, e.g., power failure, memory corruption error
    - Xv6: if we get a non-maskable interrupt --> die
    - In x86, vector 2, prevents other interrupts from executing.
- INTR: Maskable INTR
  - Ignored when [[IF]] in EFLAGS is 0
  - Enabling/disabling:
  - `sti`: set interrupt
  - `cli`: clear interrupt
- INTA:
  - Interrupt acknowledgement

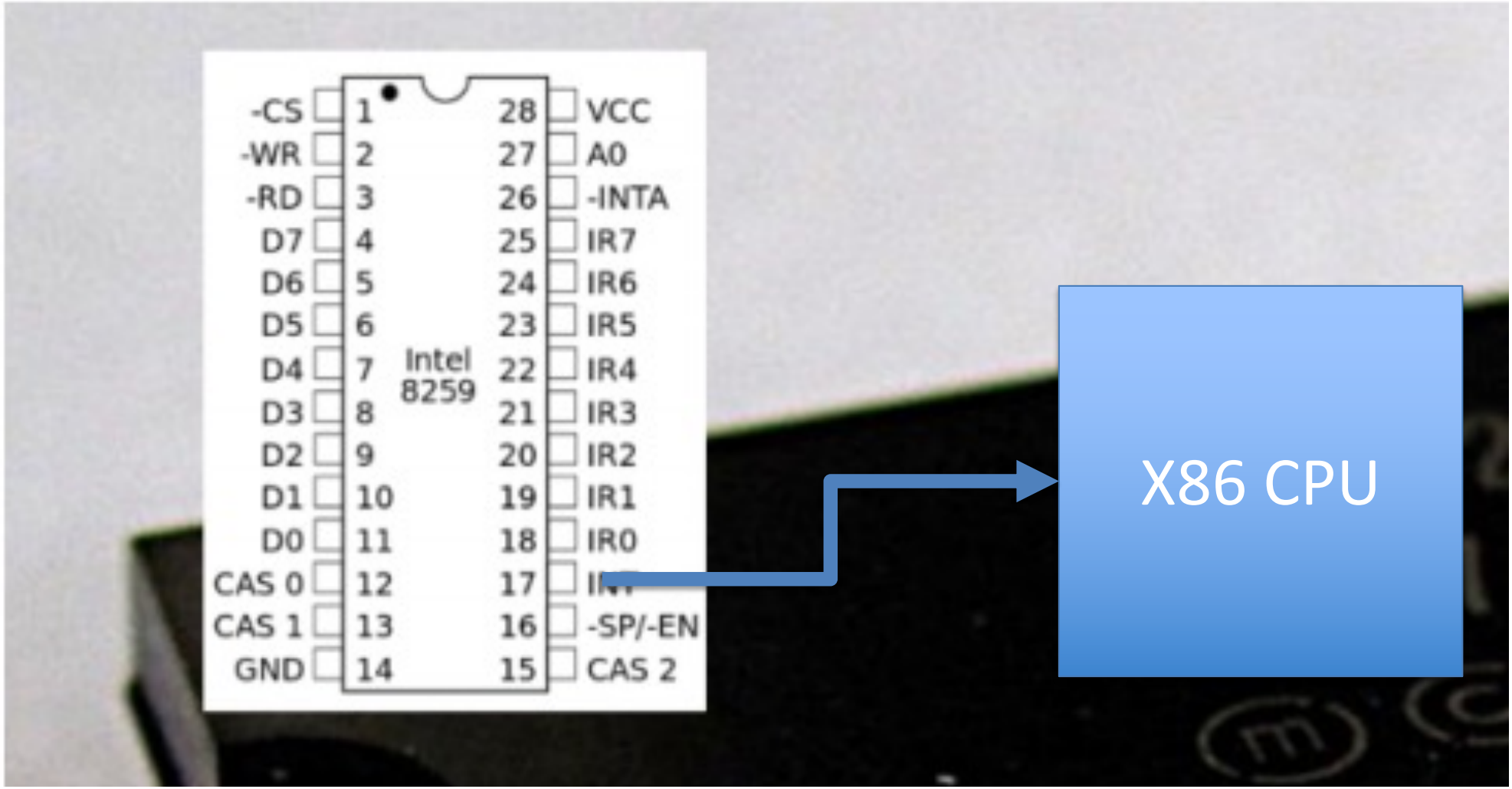
SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- How many pins?
- PIC: Programmable Interrupt Controller (8259A)



SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- How many pins?
- PIC: Programmable Interrupt Controller (8259A)



SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- "Software" interrupt: INT

INT

  - Intentionally interrupts
  - x86 provides the INT instruction
  - Invokes the interrupt handler for the vector (0-255)
  - xv6: **INT 0x40** for system calls
- Entering: int N
- Exiting: iret
  - Special instruction to return from the interrupt handler that restores the saved state before going back to userspace code

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

Georgia Tech

The **INT** instruction has the following steps:

- decide the vector number, in this case it's the 0x40 in int 0x40
- fetch the interrupt descriptor for vector 0x40 from the IDT. The CPU finds it by taking the 0x40'th 8-byte entry starting at the physical address that the IDTR CPU register points to.
- check that CPL <= DPL in the descriptor (but only if INT instruction).
- save ESP and SS in a CPU-internal register (but only if target segment selector's PL < CPL).
- load SS and ESP from TSS ("")
- push user SS ("")
- push user ESP ("")
- push user EFLAGS
- push user CS
- push user EIP
- clear some EFLAGS bits
- set CS and EIP from IDT descriptor's segment selector and offset

INTERRUPT HANDLING: HIGH LEVEL

- What interrupt?
- Is it OK to run this interrupt?
- Switch stacks (from user stack to handler/kernel stack)
- Setup our registers
  - Save registers
  - Setup registers
- Jump to kernel (set CS:IP to the handler code)

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

In xv6, x86.h defines **the "trap" frame**. It gets filled up like a stack frame from bottom to top. The first part is filled by the hardware on `int`.

```
  // below here defined by x86 hardware
  uint err;
  uint eip;
  ushort cs;
  ushort padding5;
  uint eflags;

  // below here only when crossing rings, such as from user
to kernel
  uint esp;
  ushort ss;
  ushort padding6;
```

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

In xv6, usys.S defines the interrupt for system calls.

- `vectorN` -> `alltraps` -> `trap()` -> `syscall()`

```
#define SYSCALL(name)                    \
  .globl name;                           \
  name:                                  \
    movl $SYS_ ## name, %eax;            \
    int $T_SYSCALL;                      \
    ret

SYSCALL(fork)
SYSCALL(exit)
...
```

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

In xv6, trapasm.S defines the interrupt for exiting back to user space.

- `syscall()` -> `trapret()` -> `iret`

```
.globl trapret
trapret:
  popal
  popl %gs
  popl %fs
  popl %es
  popl %ds
  addl $0x8, %esp  # trapno and errcode
  iret
```

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

In xv6, vector.S defines the interrupt vectors which jump to alltraps in trapasm.S. [See code.]

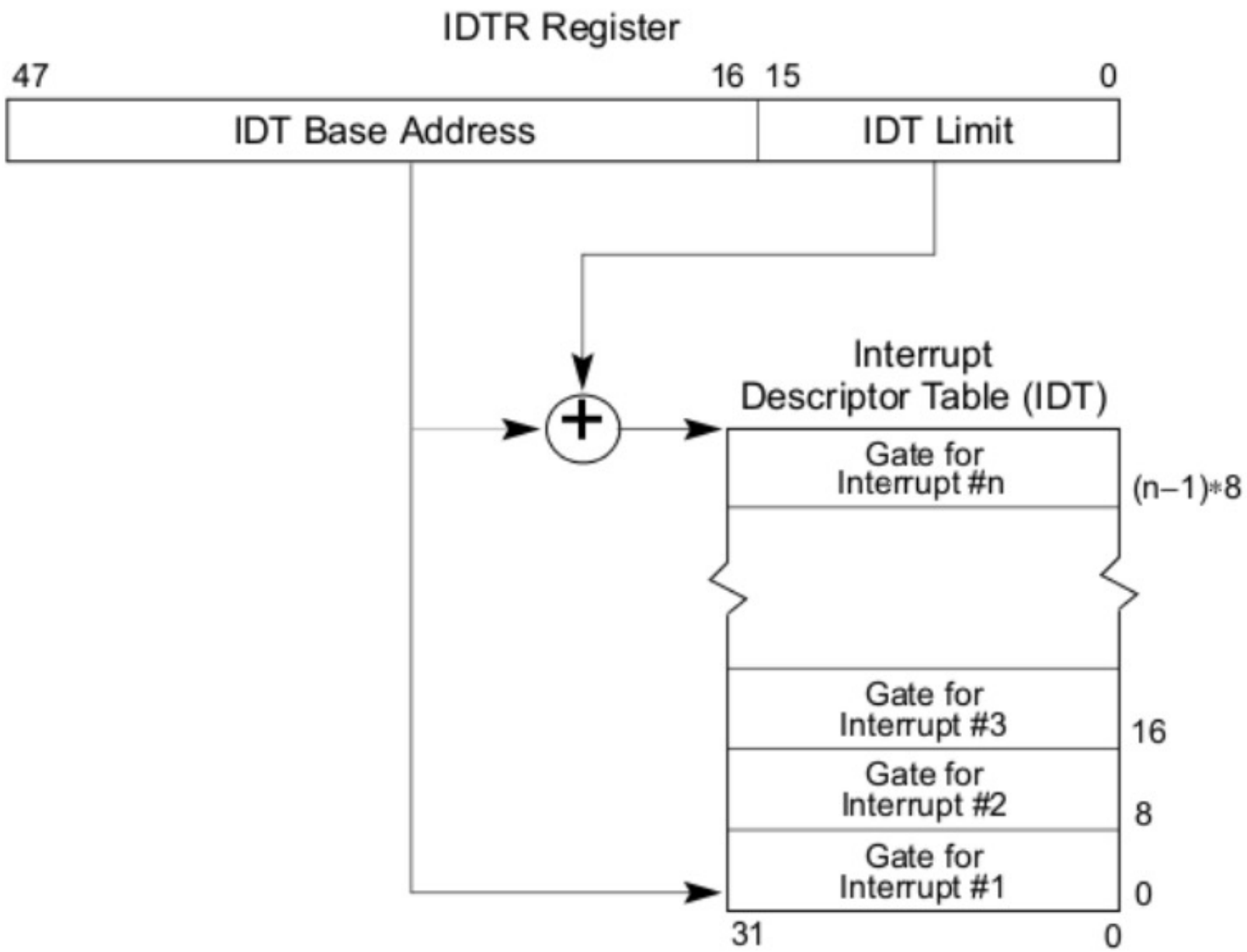
- Initial entry point in vector.S
- Kernel/src/trap.c → the vast majority of the code you will need for your labs in this class

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS

- **IDT**: Interrupt Descriptor Table
  - Table of 256 8-byte entries (similar to GDT)
- In xv6: Each specifies a protected entry-point into the kernel located anywhere in memory
- **IDTR** register (holds a ptr to current IDT)
  - Stores current IDT
- **lidt** instruction
  - Loads IDTR with address and size of the IDT
  - Takes in a linear address

Instructions to set up the IDT register

SYSTEM CALLS, INTERRUPTS, AND EXCEPTIONS



IDT INITIALIZATION

In xv6, the IDT is initialized on boot **in trap.c.**

- `main()` -> `tvinit()`

```
01  void tvinit(void)
02  {
03    int i;
04    for (i = 0; i < 256; i++)
05      SETGATE(idt[i], 0, SEG_KCODE<<3, vectors[i], 0);
06
07    // Q?
08    SETGATE(idt[T_SYSCALL], 1, SEG_KCODE<<3, \
09              vectors[T_SYSCALL], DPL_USER);
10  }
```

IDT INITIALIZATION: SETGATE

```
// Set up a normal interrupt/trap gate descriptor.
// - istrap: 1 for a trap (= exception) gate, 0 for an interrupt gate.
//   interrupt gate clears FL_IF, trap gate leaves FL_IF alone
// - sel: Code segment selector for interrupt/trap handler
// - off: Offset in code segment for interrupt/trap handler
// - dpl: Descriptor Privilege Level -
//        the privilege level required for software to invoke
//        this interrupt/trap gate explicitly using an int instruction.
#define SETGATE(gate, istrap, sel, off, d)                    \
{                                                             \
  (gate).off_15_0 = (uint)(off) & 0xffff;                    \
  (gate).cs = (sel);                                          \
  (gate).args = 0;                                            \
  (gate).rsv1 = 0;                                            \
  (gate).type = (istrap) ? STS_TG32 : STS_IG32;               \
  (gate).s = 0;                                               \
  (gate).dpl = (d);                                           \
  (gate).p = 1;                                               \
  (gate).off_31_16 = (uint)(off) >> 16;                      \
}
```

Interrupts are a pseudo concurrency!

When we program in an interruptible environment, we need to think about safety in concurrency.

CONCURRENCY

Kernel code

Lock()

Shared = 7

unlock()

Int handler

Lock()

X = shared;

Unlock()

- Can't make interrupt handler ignore the locks
- Delay the interrupt?
- Whenever I'm holding a lock, delay the interrupts